

Phần 1: KIẾN TRÚC HỆ THỐNG VÀ QUẢN LÝ BỘ NHỚ

1.1. Bản đồ bộ nhớ (Memory Map)

Chương trình không sử dụng heap động mà quy hoạch cứng các vùng nhớ trong phân khúc `.data`.

A. Vùng dữ liệu tĩnh (Static Data)

Nhãn (Label)	Kích thước	Địa chỉ (Rel)	Mô tả chi tiết
buffer	256 bytes	0x10010000	Input Buffer: Nơi lưu trữ toàn bộ chuỗi ký tự người dùng nhập vào trước khi xử lý. Kích thước 256B đủ cho dòng lệnh dài nhất.
token	64 bytes	0x10010100	Processing Unit: Vùng đệm trung gian. Khi Parser cắt chuỗi, từng từ (opcode, thanh ghi, số) sẽ được copy vào đây để phân tích.
opcode_map	~300 bytes	...	Lookup Table: Cơ sở dữ liệu chứa cặp giá trị <code><Chuỗi tên lệnh> - <Type ID></code> . Dùng để định danh lệnh.

B. Vùng giao tiếp phần cứng (MMIO)

Chương trình tương tác với các địa chỉ bộ nhớ được ánh xạ tới thiết bị ngoại vi giả lập:

Thiết bị	Thanh ghi	Địa chỉ	Bit quan trọng	Chức năng
Keyboard	CTRL	0xFFFF0000	Bit 0 (Ready)	Bằng <code>1</code> khi có phím mới được nhấn.
Keyboard	DATA	0xFFFF0004	8 bits thấp	Chứa mã ASCII của ký tự vừa nhận.

Display	CTRL	0xFFFF0008	Bit 0 (Ready)	Bằng 1 khi màn hình sẵn sàng hiển thị ký tự mới.
Display	DATA	0xFFFF000C	8 bits thấp	Ghi mã ASCII vào đây để in ra màn hình.

Phần 2: CƠ CHẾ NHẬP XUẤT DỮ LIỆU (LOW-LEVEL I/O DRIVER)

Chương trình tự viết "Driver" điều khiển thiết bị thay vì dùng hệ điều hành.

2.1. Thuật toán Polling (Vòng lặp thăm dò)

Do không sử dụng ngắt (Interrupt), CPU phải liên tục hỏi thiết bị "Có rảnh hay không?".

Logic Code:

Đoạn mã

wait_display:

```
li t0, DISPLAY_CTRL    # Tải địa chỉ điều khiển (0xFFFF0008)
lw t3, 0(t0)           # Đọc trạng thái
andi t3, t3, 0x1       # Mask lấy bit cuối cùng (Bit Ready)
beqz t3, wait_display  # Nếu bit = 0, quay lại chờ tiếp
li t0, DISPLAY_DATA    # Nếu bit = 1, thiết bị đã sẵn sàng
sw t2, 0(t0)           # Gửi dữ liệu (t2) ra màn hình
```

Phân tích: Đây là cơ chế đồng bộ (synchronous). CPU bị khóa (block) tại đây cho đến khi thiết bị ngoại vi phản hồi.

2.2. Xử lý Backspace (Trải nghiệm người dùng)

Khi phát hiện mã ASCII 8 (Backspace) hoặc 127 (Delete), chương trình thực hiện chuỗi hành động phức tạp để đánh lừa thị giác người dùng:

- Logic bộ nhớ:** Giảm con trỏ buffer s0 đi 1 đơn vị (để lần nhập sau sẽ ghi đè lên ký tự sai).
- Logic hiển thị:** Gửi chuỗi byte 8, 32, 8 ra màn hình.
 - 8: Lùi con trỏ màn hình về bên trái (đề lên ký tự sai).
 - 32 (Space): In khoảng trắng (xóa hình ảnh ký tự sai).
 - 8: Lùi con trỏ về lại vị trí vừa xóa (để sẵn sàng in ký tự mới).

Phần 3: CÁC THUẬT TOÁN PHÂN TÍCH CỐT LỖI (CORE PARSERS)

Đây là "bộ não" của chương trình, nơi xử lý logic chuỗi.

3.1. Tokenizer (Bộ tách từ) - Hàm `get_next_token`

Biến dòng lệnh `add t0, t1, 10` thành các token riêng biệt: `add`, `t0`, `t1`, `10`.

- **Đầu vào:** Con trỏ `s1` (vị trí hiện tại trong `buffer`).
- **Ký tự phân cách (Delimiters):** Space (32), Tab (9), Comma (44), Newline (10).
- **Thuật toán:**
 1. **Skip Whitespace:** Vòng lặp bỏ qua tất cả các ký tự phân cách ở đầu.
 2. **Copy Token:** Sao chép các ký tự *không phải phân cách* vào vùng nhớ `token`.
 3. **Null-Terminate:** Thêm ký tự `\0` vào cuối `token` để tạo thành chuỗi C-String chuẩn.
 4. **Cập nhật:** Di chuyển con trỏ `s1` đến vị trí tiếp theo để chuẩn bị cho lần gọi sau.

3.2. Number Parser (Bộ chuyển đổi số) - Hàm `parse_immediate`

Chuyển đổi chuỗi ký tự (String) sang số nguyên (Integer) 32-bit.

- **Hỗ trợ đa định dạng:**
 - **Decimal:** `123`, `-45`.
 - **Hexadecimal:** `0xFF`, `0x1A`.
- Thuật toán Decimal:
$$\text{Giá_trị} = (\text{Giá_trị} * 10) + (\text{Ký_tự} - '0')$$
- Thuật toán Hex:
$$\text{Giá_trị} = (\text{Giá_trị} << 4) + \text{Hex_Digit}$$

(Dịch trái 4 bit tương đương nhân 16).
- **Xử lý dấu:** Sử dụng thanh ghi cờ `t6`. Nếu phát hiện dấu - ở đầu, `t6 = 1`. Cuối hàm, nếu `t6 == 1`, thực hiện lệnh `neg` (lấy bù 2) kết quả.

3.3. Register Validator (Bộ kiểm tra thanh ghi) - Hàm `check_is_register`

Kiểm tra tính hợp lệ của toán hạng thanh ghi theo 2 luồng:

1. **Luồng 1: Tên ABI (Application Binary Interface)**
 - So sánh chuỗi token với cơ sở dữ liệu chuỗi: `zero`, `ra`, `sp`, `gp`, `tp`, `t0...t6`, `s0...s11`, `a0...a7`.
 - Sử dụng vòng lặp so sánh từng byte (`lb`, `bne`).
2. **Luồng 2: Tên phần cứng (x-register)**
 - Kiểm tra ký tự đầu tiên có phải là `x` không.
 - Parse phần số phía sau số nguyên.
 - **Range Check:** Kiểm tra số đó có thỏa mãn $0 \leq N \leq 31$ không.

Phần: HỆ THỐNG ĐỊNH TUYẾN LỆNH (INSTRUCTION DISPATCHER)

Chương trình sử dụng một bảng ánh xạ (`opcode_map`) để quyết định logic kiểm tra.

4.1. Cấu trúc Opcode Map

Bảng dữ liệu được tổ chức dưới dạng mảng các Struct:

Đoạn mã

opcode_map:

```
.word op_lw, 1    # Chuỗi "lw" -> ID 1
.word op_add, 4   # Chuỗi "add" -> ID 4
.word op_beq, 5   # Chuỗi "beq" -> ID 5
# ...
.word 0, 0        # Sentinel (Lính canh báo hiệu hết bảng)
```

4.2. Bảng mã Type ID (Instruction Types)

Chương trình định nghĩa các ID nội bộ để tái sử dụng mã kiểm tra:

- **ID 1 (Type Load):** Kiểm tra **Reg, Address**.
- **ID 10 (Type Store):** Kiểm tra **Reg, Address** (Logic giống Load).
- **ID 2 (Type U):** Kiểm tra **Reg, Imm20**.
- **ID 3 (Type I):** Kiểm tra **Reg, Reg, Imm12**.
- **ID 7 (Type Shift):** Kiểm tra **Reg, Reg, Imm5**.
- **ID 4 (Type R):** Kiểm tra **Reg, Reg, Reg**.
- **ID 5 (Type B):** Kiểm tra **Reg, Reg, Label**.
- **ID 6 (Type J):** Kiểm tra **Reg, Label**.

Phần 5: PHÂN TÍCH CHI TIẾT LOGIC KIỂM TRA (VALIDATION LOGIC)

Đây là phần quan trọng nhất, mô tả cách chương trình xác định đúng/sai cho từng loại lệnh.

5.1. Xử lý Lệnh R-Type (**add, sub, or...**)

- **Cú pháp:** **OP rd, rs1, rs2**
- **Quy trình:**
 1. Gọi **check_reg_op1**: Xác thực **rd**.
 2. Gọi **check_reg_op2**: Xác thực **rs1**.
 3. Gọi **check_reg_op3**: Xác thực **rs2**.
- **Ví dụ Lỗi:** **add t0, 1, t2** -> Lỗi tại Operand 2 vì **1** không phải thanh ghi.

5.2. Xử lý Lệnh Load/Store (**lw, sw...**)

- **Cú pháp:** **OP reg, offset(base)**
- **Quy trình:**
 1. Gọi **check_reg_op1**.
 2. Gọi **check_address_op2**:
 - Tách phần trước (:): Gọi **check_is_12bit_imm**. Offset phải nằm trong khoảng [-2048, 2047].
 - Tách phần trong (): Gọi **check_is_register**. Base phải là thanh ghi hợp lệ.

- Ví dụ Lỗi: `lw t0, 4000(sp)` -> Lỗi Offset vì $4000 > 2047$.

5.3. Xử lý Lệnh I-Type (**addi**, **andi**...)

- Cú pháp: `OP rd, rs1, imm`
- Quy trình:
 1. Kiểm tra 2 thanh ghi đầu.
 2. Gọi `check_12bit_imm_op3`.
- Đặc biệt: Với lệnh `jalr` (Jump And Link Register), cấu trúc cũng là I-Type nên được xếp vào nhóm này.

5.4. Xử lý Lệnh Shift (**slli**, **srai**...)

- Cú pháp: `OP rd, rs1, shamt`
- Quy trình:
 - Kiểm tra 2 thanh ghi đầu.
 - Gọi `check_5bit_imm_op3`: Kiểm tra Shift Amount.
 - Điều kiện: $0 \leq \text{shamt} \leq 31$.
- Lý do: Kiến trúc RV32 chỉ có 32 bit, không thể dịch quá 31 vị trí.

5.5. Xử lý Lệnh Branch (**beq**, **bne**...)

- Cú pháp: `OP rs1, rs2, label`
- Quy trình:
 1. Kiểm tra 2 thanh ghi so sánh.
 2. Gọi `check_label_op3`: Kiểm tra nhãn nhảy.
 - Quy tắc nhãn: Bắt đầu bằng chữ hoặc `_`. Không chứa ký tự đặc biệt.
 - Validation: Tên nhãn không được trùng với từ khóa dành riêng (Opcode). Ví dụ: `beq t0, t1, lw` là sai cú pháp.

Phần 6: HỆ THỐNG QUẢN LÝ LỖI (ERROR HANDLING)

Chương trình áp dụng nguyên tắc **Fail-Fast** (Dừng ngay khi gặp lỗi).

6.1. Mã lỗi

Các thông báo lỗi được định nghĩa sẵn trong `.data`:

- `msg_err_op`: "Error: Invalid or unsupported opcode."
- `msg_err_op1`: "Error: Invalid Operand 1."
- `msg_err_op2`: "Error: Invalid Operand 2."
- `msg_err_op3`: "Error: Invalid Operand 3."

6.2. Cơ chế nhảy (Control Flow)

Mỗi hàm kiểm tra (Checker) trả về kết quả trong thanh ghi `a0`:

- `a0 = 1`: Hợp lệ.
- `a0 = 0`: Không hợp lệ.

Sau mỗi lần gọi hàm kiểm tra, chương trình kiểm tra ngay `a0`:

Đoạn mã

jal check_is_register

beqz a0, e_op1 # Nếu a0 == 0, nhảy ngay lập tức đến nhãn báo lỗi e_op1